

LucidForm: A Deterministically Gated Architecture for Safe LLM-Assisted Form Filling

Shubh Pratap Singh	Yashvardhan Singh Sarangdevot	Lakshay Gupta	Kanishka Jain	Sanjith
School of Computer Science and Engineering	School of Computer Science and Engineering	School of Computer Science and Engineering	School of Computer Science and Engineering	School of Computer Science and Engineering
VIT Bhopal University	VIT Bhopal University	VIT Bhopal University	VIT Bhopal University	VIT Bhopal University
Sehore – 466114, India	Sehore – 466114, India	Sehore – 466114, India	Sehore – 466114, India	Sehore – 466114, India
shubh.23bce10910@vitbhopal.ac.in	yashvardhan.23bce10849@vitbhopal.ac.in	lakshay.23bce10957@vitbhopal.ac.in	kanishka.23bce10457@vitbhopal.ac.in	sanjith.23bce11149@vitbhopal.ac.in

Abstract—Government and banking forms in India — KYC, loan applications, insurance claims — remain a barrier for users who cannot reliably read them, and large language models are an obvious but dangerous fix: an LLM that both interprets a user's speech and writes the resulting value into a legal document gives an unreviewable failure mode to exactly the users least able to review it. We present LucidForm, a form-filling assistant built on the premise that the model must never hold write authority. Every candidate value produced by the LLM passes through a deterministic, non-LLM validation gate — format, checksum, and cross-field rules — is read back to the user in plain language, and is committed only on an explicit, whitelisted affirmation. We enforce this separation three ways: structurally, through a private write path that re-verifies a cryptographic fingerprint at commit time; through a capability-guarded receipt object that can only be constructed by the confirmation step; and architecturally, through a static-analysis test that fails the build if either guarantee is bypassed. We evaluate the validation gate and the confirmation parser against hand-authored adversarial suites (56 and 49 cases respectively, spanning eleven and seven failure categories) and report a layered-defense result over three synthetic personas: of six erroneous values proposed during full-pipeline sessions, the gate rejected five and the read-back caught the sixth — a well-formed but misheard value no deterministic check could have flagged — with zero values committed incorrectly. We report these figures plainly as preliminary, from a small synthetic corpus, and state explicitly what remains unmeasured: live-model extraction accuracy at scale, Hindi and voice-channel performance, and behavior on forms other than the one prototype we built.

Keywords—Accessible Computing, LLM Guardrails, Human-in-the-Loop, Structured Output, Form Understanding, Deterministic Validation, Digital Identity, Conversational AI

I. INTRODUCTION

Government and financial institutions in India process Know Your Customer (KYC) forms, loan applications, and insurance claims at a scale that assumes the applicant can read the form, understand its jargon, and correctly transcribe identifiers such as a PAN or Aadhaar number onto it. This assumption fails for a large population: low-literacy users, elderly applicants unfamiliar with financial terminology,

and visually-impaired users for whom the form itself is inaccessible. Existing remedies — a literate relative, a paid form-filling agent, a bank branch visit — are inconsistent, sometimes exploitative, and unavailable to a user acting alone.

A conversational assistant is a natural fix, and large language models make one straightforward to build: transcribe the user's speech, ask a model to read the transcript and the form together, and let it produce a filled document. This design is rejected here, deliberately and on a specific ground. It collapses two roles that must stay separate — interpreting what a user meant, and deciding that a value is correct and may be recorded — into a single component that performs both without external check. A model that infers a struck digit, resolves an ambiguous answer in its own favor, or is manipulated by adversarial phrasing embedded in the user's utterance writes that value directly into a document with legal standing. For a sighted, literate user this failure is at least detectable on re-reading the form. For the population this system targets, it is often not detectable at all: a user who cannot read the form cannot audit what was written into it. The single-model design therefore fails hardest for exactly the user it is meant to serve.

LucidForm is built around the opposite premise: the language model never writes a value into the form. There is exactly one code path capable of mutating form state, and the model has no way to reach it. A candidate value must instead traverse five stages, in a fixed order, before it can be committed:

1. **Form parsing** — field identity, type, and structural constraints (e.g. maximum length) are read once from the target document, ahead of any conversation.
2. **Conversational orchestration** — one field is requested at a time, in plain language, with jargon explained on request rather than assumed.
3. **Extraction** — the user's free-text or spoken response is converted by the model into a typed, frozen Candidate.

A candidate is explicitly a proposal, not a value the system holds.

4. **Validation** — a deterministic component with no model dependency accepts or rejects the candidate against format, checksum, and cross-field rules, returning exactly one reason code on rejection.
5. **Read-back and confirmation** — the validated value is stated back to the user in a form they can audibly verify; only an explicit, unambiguous affirmation produces a commit.

This paper’s contribution is the architecture that makes step 4 load-bearing rather than decorative — three independent mechanisms (structural, capability-based, and static-analysis-enforced) that make “the model cannot write a value” a property the system can be tested for, rather than a design intention that later code may quietly violate — together with an adversarial evaluation methodology built to falsify that property rather than merely demonstrate it. The system is a working prototype, not a finished study: it currently supports one form type (an individual KYC-equivalent form), English text input at full pipeline maturity, and is mid-integration on a Hindi and voice channel. Section VII states these limits without qualification, because a system whose safety claim rests on structural guarantees should not need its scope understated to look complete.

The remainder of this paper is organized as follows. Section II situates LucidForm against prior work on accessible form-filling, LLM guardrails, and speech recognition on structured identifiers. Section III describes the architecture and its enforcement mechanisms in detail. Section IV describes the methodology — form representation, synthetic data construction, and the adversarial and evaluation harnesses built alongside the system rather than after it. Section V gives the governing algorithms in pseudocode. Section VI reports preliminary results from the current prototype. Section VII discusses what the results do and do not show. Section VIII concludes and lays out the remaining phases of the work.

II. RELATED WORK

Conversational and voice-based form-filling for low-literacy users in India. The closest prior system is FormBharo [1], a voice agent built with an NGO partner (ARMMAN) to enroll mothers in a maternal-health program over the phone in Hindi. FormBharo combines an LLM with rule-based validation under real latency and cost constraints, and its released benchmark shows that component-level accuracy figures do not predict end-to-end form-completion success under real acoustic noise — a finding consistent with the position taken here, that an isolated extraction-accuracy number is the wrong quantity to optimize or report. LucidForm differs in emphasis rather than in premise: FormBharo’s rule-based layer is one component among several optimized for a live deployment;

this work isolates the validation-and-confirmation boundary as the object of study and evaluates it adversarially, independent of a specific deployment’s latency budget. A comparative study of form-based versus conversational public-service interfaces in India [2] reports that low-income and low-literacy participants engage more successfully with a conversational, voice-first design than a traditional form, supporting the premise that conversation is the right interaction mode without addressing what happens when the model that holds the conversation is also asked to guarantee correctness. Related work on automating government form completion for elderly and monolingual users [3] and on bilingual conversational assistants for banking [4], [5] establishes accessibility-driven form and financial-service automation as an active area, generally without a stated mechanism preventing an unreviewed model output from being recorded.

Guardrails and human-in-the-loop control of LLM output. NeMo Guardrails [6] introduced programmable “rails” that constrain a dialogue system’s behavior through externally defined rules rather than prompt instruction alone, establishing that a deterministic control layer around a language model is both practical and necessary for controllable behavior — the same principle this work applies specifically to a commit decision rather than to dialogue flow generally. More recent guardrail-pipeline proposals [7], [8] and per-field uncertainty estimators for structured LLM output [9] pursue a related goal — flagging or filtering untrustworthy model output for human review — but generally operate as a confidence filter layered on top of the model’s output rather than as a control that the model has no channel to bypass. LucidForm’s validation gate is deliberately narrower and stronger than a confidence filter: it is enforced through the type system and the write path itself, verified by static analysis, and evaluated by trying to defeat it rather than by measuring how often it agrees with a human reviewer.

Automatic speech recognition on identifiers versus prose. A recognizer’s word error rate is a poor predictor of whether it preserves an exact identifier. VoiceCodeBench [10] evaluates twelve ASR systems against 1,482 audited structured-entity targets (identifiers, codes, quantities) and finds the strongest system achieves only 68.7% task success at recovering them exactly, with correlation between word error rate and exact-match success as low as -0.73 — i.e., a transcript can read fluently while still corrupting the one value that matters. Earlier work from Google’s speech team [11] shows end-to-end ASR models degrade specifically on numeric sequences relative to prose, and demonstrates substantial recovery through targeted training rather than general model improvement. This literature motivates a structural design choice in LucidForm rather than merely a claim: identifiers are read back character-by-character and digit-by-digit specifically because they are the class of value

ASR is known to corrupt silently, in a form that remains well-formed and passes every deterministic check.

Structured extraction from forms, and slot-filling with confirmation. FormNet [12], the form-field extraction model of Majumder et al. [13], and Form2Seq [14] address the separate problem of recovering field structure from a form document itself — relevant to this work's form-parsing stage but orthogonal to its validation-and-commit contribution, since LucidForm's target document already exposes machine-readable field structure through the PDF AcroForm standard. Slot-filling dialogue systems have long distinguished extracting a value from committing it: belief-tracking work in spoken dialogue systems [15] and empirical studies of clarification and confirmation strategies [16] show that explicit confirmation measurably improves the precision of values a dialogue system records, which is the confirmation step's role here, made a hard requirement rather than a strategy that improves an average.

Checksum design for identity numbers. The Aadhaar identifier's check digit uses the Verhoeff algorithm [17], the first decimal check-digit scheme proven to detect all single-digit substitution errors and all adjacent-transposition errors, adopted for Aadhaar-related validation as a matter of banking-sector practice [18]. The 12-digit, non-intelligent (semantic-free) design of the Aadhaar number itself is documented in the UID numbering scheme white paper [19] and the UIDAI's own demographic data standards report [20]. LucidForm implements Verhoeff rather than the more commonly implemented Luhn algorithm because Luhn accepts a class of transposition errors Verhoeff is designed to catch — a distinction with no visible effect until it is the one digit that matters.

III. SYSTEM ARCHITECTURE

A. The five-stage pipeline

Fig. 1 shows the path a value must traverse. Form parsing runs once, ahead of any session, and produces a typed field specification for every field on the target form: identifier, plain-language prompt, jargon gloss, data type, and structural constraints. The orchestrator asks about one field at a time and performs no validation of its own; it forwards whatever it receives to extraction and forwards whatever extraction and validation report back to the user, unmodified, so that there is exactly one place in the system that explains why a value was rejected. Extraction converts the user's utterance into a frozen Candidate object using a model call whose response is constrained to a fixed schema (Section III-C) — a proposal, never a value the system holds as fact. Validation is the first point at which the model's judgement can be overridden and is described in Section III-B. Read-back and confirmation, described in Section III-D, are the last stage before commit and the only stage capable of catching an error that is well-formed and therefore invisible to validation.

Parser → Orchestrator → Extraction (LLM) → [VALIDATION GATE] → Read-back → Confirm → Commit

Fig. 1. The five-stage pipeline. No value reaches the form without traversing every stage.

B. The validation gate and its rejection taxonomy

The gate is deterministic: format regular expressions, the Verhoeff checksum for Aadhaar-format identifiers, a cross-check between a PAN-format identifier's fifth character and the confirmed surname, and a postal-code-to-state lookup. It imports no language-model client, directly or transitively; this is enforced by the static-analysis test described in Section III-E rather than left as a convention. Every rejection carries exactly one reason code from a closed, ordered taxonomy (Table I), evaluated in a fixed sequence so that a value failing more than one check is reported by its most severe defect rather than an arbitrary one.

TABLE I. REJECTION TAXONOMY AND CHECK ORDER

Order	Code	Condition
1	EMPTY	No value, or only invisible/whitespace characters
2	TYPE_MISMATCH	Value is the wrong length for its field
3	FORMAT	Right length, wrong shape (fails the field's pattern)
4	ENUM	Not one of a fixed set of permitted options
5	CHECKSUM	Well-formed but arithmetically invalid
6	RANGE	Well-formed but outside a permitted domain (e.g. a future date)
7	CROSS_FIELD	Contradicts an already-confirmed value on another field
8	AMBIGUOUS_EXTRACTION	The extractor itself reported more than one plausible reading
9	LOW_CONFIDENCE	The extractor reported low confidence in its own reading

Structural checks (1–7) are ordered ahead of extraction-quality checks (8–9) deliberately: a value that is both malformed and reported with low confidence is a malformed value, and reporting it as a confidence problem would substitute a weaker finding for the actual defect. Normalization — removing separators, folding case, resolving a country-code prefix that leaves exactly the expected digit count — runs before every check and never repairs a value that is actually wrong: a short identifier is never padded, a failed checksum is never corrected, and a value in an unlisted script is never transliterated, because doing so would commit a value inferred by the system rather than one the user stated. Cross-field checks read a second field's value only once that field has itself been

confirmed, so an unconfirmed value can never indirectly influence which values the gate accepts for another field.

C. Extraction as a structurally bounded model call

The model's output is constrained to a fixed schema with six fields — an intent classification (stating a value, asking a question, declining, or unrecoverable), a proposed value, a verbatim quotation of the utterance region the value was drawn from, a self-reported confidence, an ambiguity flag, and a list of alternative readings. The schema has no field in which the model could assert that a value is valid or should be written; a prompt-injection attempt embedded in the user's utterance therefore has no channel through which to act, independent of how the prompt is worded, because the failure mode it would need to produce cannot be expressed in the response format.

Two checks run on the model's output without invoking a model a second time. Grounding locates the model's quoted span in the original utterance; a value whose quotation cannot be found there is treated as ungrounded regardless of the confidence reported, since it was not demonstrably derived from anything the user said. Grounding proves anchorage to the utterance, not correctness of the reading — a digit sequence can be quoted faithfully and still be a mistranscription of what was spoken, which the gate and read-back exist to catch. Confidence clamping allows a deterministic finding to lower the model's self-reported confidence, and only to lower it: an ungrounded extraction is forced to zero confidence and marked ambiguous, so it is rejected by both the gate's confidence and ambiguity checks and stays rejected if either threshold is later relaxed.

D. Read-back and the confirmation whitelist

For a user who cannot see the form, the read-back is not a courtesy summary — it is the only representation of the value they will ever receive, and the confirmation step is meaningless if that representation is not something the user can actually check by ear. Presentation is type-dependent: identifiers are read character by character in grouped, paused sequences, because a synthesizer reading a twelve-digit identifier as a single number has silently regrouped it in a way the listener cannot verify against a physical card; punctuation inside an address is spoken as a word, because a symbol a synthesizer silently skips is a symbol the user cannot confirm.

The confirmation parser is a whitelist evaluated against the entirety of the user's utterance, not a search for an affirmative token within it. This choice is motivated by a specific and realistic failure: the utterance “yes, I know that's wrong” contains an affirmative token and is, in fact, a rejection; “yesterday I gave you the wrong number” contains one embedded in an unrelated word. A substring-based check commits an unagreed value in both cases, for a user who by construction cannot inspect the resulting document afterward to notice. Negation and hedging cues

defeat confirmation before the whitelist is consulted, and acknowledgement tokens (e.g., “okay”, “acha”) are deliberately excluded — they indicate the user heard the read-back, not that they agree with it. This asymmetry is intentional: a false negative costs a re-ask and no incorrect commit; a false positive commits a value into a legal document the user did not agree to. These costs are not comparable, and the parser's design reflects that rather than optimizing for a single accuracy figure.

E. Three independent enforcement mechanisms

The guarantee that a value cannot be committed without traversing validation and explicit confirmation is not stated once but enforced three separate times, each covering a failure mode the others do not.

1) Structural. Form state exposes no public setter. The single write operation accepts a confirmation receipt and independently re-verifies, at the write site, that the receipt was issued for the exact field and candidate being committed — including recomputing a cryptographic fingerprint from the candidate rather than trusting one carried on the receipt — that the validation report it references passed, and that affirmation was explicit. A receipt issued for one value cannot be used to authorize committing a different one.

2) Capability. The receipt object cannot be constructed anywhere outside the confirmation module; construction requires a sentinel value held privately there, checked via a runtime inspection of the calling stack frame. This converts “only the confirmation step may issue a receipt” from a convention every future contributor must remember into a property enforced by the type itself.

3) Architectural. A static-analysis test parses the codebase's abstract syntax tree and fails the build if the validation or form-state modules import the model client, if a receipt is constructed outside the confirmation module, or if form state's private fields are accessed elsewhere. This is the mechanism that catches deliberate circumvention, since it runs at review time rather than relying on a runtime guard being reached at all.

All three mechanisms were verified to detect violations, not merely to pass in their absence: each rule was deliberately violated in a throwaway change, confirmed to fail the build with the offending file and line identified, and reverted. A safety test that has never been observed to fail is not evidence that it works.

IV. METHODOLOGY

A. Form representation

The target document is a fillable PDF (AcroForm) reproducing the field set of a standard individual KYC form — name, date of birth, gender, parent's name, PAN, Aadhaar, address, postal code, mobile number, email,

occupation, and income band — generated programmatically rather than sourced from an official document, to keep the corpus reproducible and avoid redistributing a third-party artifact. Field identity, widget type, and structural constraints such as maximum length are read directly from the document's field dictionary; this required walking widget annotations directly, since the PDF library's higher-level field accessor was found to silently drop the maximum-length attribute, an error that is otherwise invisible because the system falls back to whatever a separately authored constraint file asserts. Semantic constraints — that a field holds a PAN, that a PAN's fifth character encodes a surname initial — are not recoverable from PDF structure and are supplied by a human-authored overlay of approximately one hundred lines per form type. This division is stated explicitly rather than presented as automatic form understanding: the contribution under evaluation is the gating architecture, and the parser is generic over AcroForm documents, so substituting a different form is a matter of authoring a new overlay rather than modifying code.

B. Synthetic data and its construction

All identities, PAN and Aadhaar numbers, addresses, and phone numbers used in this work are fabricated. No real personal data, filled form, or identity document was used at any stage. Synthetic Aadhaar-format numbers are constructed to be Verhoeff-valid by algorithmic generation rather than sourced from any real number, so that the checksum path is genuinely exercised without using or requesting real identifiers.

Three synthetic personas were authored to span the conditions the system targets: a low-literacy user who reads identifiers aloud digit by digit; a visually-impaired user for whom the spoken read-back is the sole channel through which an error can be caught; and a Hindi-English code-mixing user who requests a field's meaning before answering it. Persona utterances are hand-authored rather than model-generated, because the input distribution that matters for this evaluation — an identifier spelled character by character, a value embedded in surrounding conversational text, a request for clarification arriving where a value was expected — is a property of the target user population, and generating it with the same model under evaluation would make the evaluation partly circular. Each persona's identifiers are checked by test against the same constraints the gate itself enforces, so that an invalid ground-truth value cannot inflate the reported false-positive rate with a corpus error rather than a gate error.

C. Adversarial evaluation corpora

The validation gate and confirmation parser were each evaluated against a hand-authored adversarial corpus written before the component under test, so that a case the component fails on first run is a genuine finding rather than

a specification retrofitted to the implementation's existing behavior.

The gate corpus comprises 56 cases across eleven categories: single-digit checksum errors, visually confusable character substitutions of the kind speech recognition produces, numeric transcription errors, Hindi-English numeral mixing, prompt-injection strings embedded in an utterance, cross-field contradictions between individually valid values, empty and invisible-character input, over-length input, type coercion, and values well-formed for a different field. Approximately one fifth of the cases are controls that must be accepted — a correct checksum beside three corrupted variants, a correctly paired postal code beside a mismatched one — because a component that rejects every input satisfies a rejection-only suite completely while being useless; these controls are what the reported false-positive rate is measured against.

The confirmation corpus comprises 49 cases across seven categories, including two specifically adversarial to a naive implementation: affirmation spoofing, where an utterance contains an affirmative token while withholding consent (“yes, I know that's wrong”), and substring traps, where an affirmative token appears inside an unrelated word. Thirteen cases are controls that must confirm, again to guard against a parser that satisfies the suite by refusing everything. A separate test implements the naive substring-search parser directly and confirms that it fails at least eight of the suite's rejection cases, establishing that the suite discriminates between the two designs rather than being trivially satisfiable by either.

D. Evaluation harness and its honesty constraints

Every pipeline stage emits an append-only, timestamped event to a log from the first phase of implementation, before any stage that would populate it existed, so that instrumentation was never retrofitted around a result that had already been observed. Analysis is a separate step that reads recorded logs and re-runs no sessions, so a change in how a measure is computed can be re-applied to data already collected.

Two distinctions are enforced throughout the harness because violating either would produce a number that looks like a result but is not. First, offline replay against a corpus of expected model responses (derived from the same ground truth an accuracy figure would be scored against) exercises every stage of the pipeline deterministically and will detect a regression anywhere in it, but cannot measure extraction accuracy — that figure is necessarily perfect by construction and is reported as such, with the report's own output stating that it carries no information. Extraction accuracy is consequently a claim reserved for live-model evaluation, run through the same replay driver against the real API. Second, gate recall is never reported without the gate's false-positive rate alongside it: a validator that rejects

every input achieves perfect recall while being unusable, and the false-positive rate is the measure that distinguishes a discriminating gate from an obstructive one — a distinction the harness treats as a methodological requirement rather than an optional addition, since it is the measure most often absent from comparable reported systems.

V. ALGORITHM AND PSEUDOCODE

Algorithm 1. Commit invariant (FormState.commit)

```
function commit(field_id, candidate, receipt):
    assert receipt.field_id == field_id
    assert receipt.candidate_id == candidate.candidate_id
    assert receipt.validation.status == PASS
    assert receipt.affirmation.explicit == True
    assert candidate.value is not empty
    expected_fp = sha256(field_id ||
normalize(candidate.value)
|| candidate.candidate_id)
    assert receipt.candidate_fingerprint == expected_fp
    # every assertion above is independently re-checked here;
    # none is assumed established by an earlier stage
    _values[field_id] = candidate.value # the ONLY
write to _values
    emit(COMMIT, field_id, candidate, receipt)
    return

on AssertionError:
    emit(SILENT_WRITE_BLOCKED, field_id, candidate)
    raise SilentWriteBlocked
```

Algorithm 2. Validation gate (fixed check order)

```
function validate(candidate, confirmed_fields,
field_spec):
    value = normalize(candidate.value, field_spec)
    for check in [EMPTY, TYPE_MISMATCH, FORMAT, ENUM,
CHECKSUM,
RANGE, CROSS_FIELD,
AMBIGUOUS_EXTRACTION,
LOW_CONFIDENCE]:
        result = run(check, value, candidate,
confirmed_fields, field_spec)
        if result is FAIL:
            return ValidationReport(status=REJECT,
reason=check,
detail=result.detail)
    return ValidationReport(status=PASS,
normalized_value=value)
```

Algorithm 3. Confirmation as a whitelist over the whole utterance

```
function parse_affirmation(utterance):
    normalized =
strip_filler_and_hedging(casefold(utterance.strip()))
    if normalized in NEGATION_PATTERNS or normalized in
HEDGE_PATTERNS:
        return Affirmation(explicit=False,
basis="negation_or_hedge")
    if normalized in AFFIRMATION_WHITELIST: # exact
match, NOT substring
        return Affirmation(explicit=True,
basis="whitelist_match")
    return Affirmation(explicit=False, basis="no_match")
```

VI. PRELIMINARY RESULTS

These results are reported from the current prototype and are explicitly preliminary: they are drawn from a synthetic corpus of three personas and an offline (non-live-model) pipeline configuration, for the reasons given in Section IV-D. They demonstrate that the enforcement mechanisms and the layered defense they compose behave as designed on the cases constructed to test them; they are not a claim about extraction accuracy or usability at scale.

A. Test suite. The full test suite — unit tests for every module, both adversarial corpora, the static-analysis safety test, and end-to-end persona-driven sessions — comprises 420 tests, of which 416 pass and 4 are skipped (live-API tests that require credentials not present in this evaluation environment). No test fails.

B. Adversarial suite outcomes. Every one of the 56 gate cases and 49 confirmation cases produces its declared expected outcome (reject with the declared reason code, or accept for a control case). The naive substring-search confirmation parser, implemented separately for comparison, fails 8 of the 49 cases outright — concrete evidence that the suite discriminates a correct implementation from a plausible incorrect one, rather than being satisfiable by either.

C. Layered defense over full-pipeline sessions. Three complete, persona-driven sessions were run against the full text pipeline (parsing through commit), producing six proposed values that did not match the persona's ground truth. Table II shows where each was stopped.

TABLE II. WHERE ERRONEOUS VALUES WERE STOPPED (n = 6, THREE SESSIONS)

Layer	Count
Rejected by the validation gate	5
Denied at read-back (well-formed; gate could not detect it)	1
Committed incorrectly	0

Gate recall alone over this sample is 83% (5/6); the layered system stopped all six. The gate's false-positive rate over the same sessions was 0% — no correct value was rejected. The one error the gate could not catch was a city name misrecognized as a different, real city: correct in type, length, and format, and consistent with every other confirmed field, so no deterministic rule had grounds to reject it. It was caught only because the simulated user, hearing the value read back, recognized it as wrong — the specific case this architecture's read-back stage exists to address, and the reason gate recall alone is reported alongside the layered figure rather than in its place.

We emphasize what these numbers do not show. A sample of six erroneous values from three sessions is too small to support a quoted percentage as a general claim about gate recall, and the offline configuration used here (Section IV-D) makes any latency or extraction-accuracy

figure computed alongside these results uninformative by construction; both are omitted rather than reported with a caveat that could be dropped by a later reader.

VII. DISCUSSION

What is demonstrated. The results in Section VI support a narrower and more specific claim than “the system fills forms accurately”: that a deterministic validation-and-confirmation boundary can be built as a testable, falsifiable property of a codebase rather than an assumed consequence of careful prompting, and that doing so catches a class of error — a value that is well-formed but simply not what the user said — that a purely rule-based gate cannot, and that a purely confidence-based filter would not reliably surface either. This is the property comparable systems in Section II generally do not report a mechanism for: FormBharo’s rule-based layer improves reliability under real deployment constraints but is not evaluated as an architectural guarantee; guardrail toolkits [6]–[9] filter or flag model output rather than making a commit path structurally unreachable from the model. The contribution here is narrower in scope than either — one commit boundary, evaluated adversarially — and that narrowness is deliberate.

Limitations, stated plainly. The evaluation corpus is small: three personas and six erroneous proposals is sufficient to demonstrate that the layered mechanism functions, not to estimate its recall as a stable statistic. The prototype supports one form type; a different form’s semantic overlay has not been authored or tested, so generality across form types is an untested claim rather than an established one. Extraction accuracy against a live model has not yet been measured in this evaluation environment, for a mundane reason (no API credentials were available at evaluation time) rather than a methodological one, and the offline figures that exist are stated in Section VI as uninformative rather than substituted for the missing measurement. The Hindi language channel is implemented for prompts and read-back but has an open design question — whether digits should be spoken as Hindi or English number words to match how the target population actually reads them — that requires input from native speakers of the target dialect rather than an internal decision, and is deferred rather than guessed at. The voice channel (automatic speech recognition and speech synthesis) is implemented behind the same channel-abstraction boundary used for text but has not yet been evaluated with real speech; a round-trip probe exists to measure identifier survival through synthesis and recognition specifically, and is reported only once run against a real acoustic backend rather than a simulated one, to avoid the same circularity problem described for offline extraction.

On the frame guard. The capability mechanism described in Section III-E2 inspects the calling stack frame at runtime and is explicitly documented as defending

against an accidental violation — a helper function drifting into the wrong module during a refactor — rather than a determined attempt to bypass it via direct manipulation of Python’s frame or dataclass internals, which frame inspection cannot prevent. The architectural (static-analysis) mechanism is the one relied upon for deliberate circumvention, since it operates at review time rather than at runtime.

VIII. CONCLUSION AND FUTURE WORK

We have presented LucidForm, a form-filling assistant for low-literacy and visually-impaired users built around a single architectural commitment: a large language model may interpret a user’s speech but may never hold the authority to write a value into a form. We enforce this through a five-stage pipeline whose validation and confirmation stages are deterministic, and through three independent mechanisms — a re-verifying write path, a capability-guarded receipt, and a static-analysis test — that make the guarantee checkable rather than asserted. Adversarial evaluation of the validation gate and confirmation parser, and a layered-defense measurement over synthetic persona sessions, show the mechanism functioning as designed on a small but deliberately constructed corpus, while stopping short of claims about accuracy or usability the current evidence does not support.

Immediate future work follows the phases already scoped for this prototype: resolving the open design question on Hindi numeral pronunciation and completing Hindi-language evaluation; completing and evaluating the voice channel against real speech, including the identifier-survival probe described in Section VII; running the live-model extraction-accuracy evaluation the offline harness was built to support; and expanding the synthetic persona corpus, since six erroneous proposals is a foundation for the methodology, not a sufficient sample for the recall and false-positive figures it produces to be quoted as stable results. Longer term, generalizing the architecture to additional form types depends on authoring and testing further semantic overlays, and an accessible web interface — deferred behind the same channel abstraction used for voice — is scoped as optional future work should the core pipeline’s evaluation leave time for it.

REFERENCES

- [1] A. Dalmia, S. Midha, and J. Doshi, “FormBharo: Designing and Evaluating a Voice Agent for Conversational Form Filling in Rural India,” arXiv:2608.06027, 2026.
- [2] “First Impressions from Comparing Form-Based and Conversational Interfaces for Public Service Access in India,” in Proc. HCI+NLP Workshop, ACL, 2025.
- [3] “Automated government form filling for aged and monolingual people using interactive tool,” Disability and Rehabilitation: Assistive Technology, 2023.
- [4] “AI-enhanced bilingual banking assistant,” Scientific Reports, 2025.

- [5] “Multilingual Conversational AI for Financial Assistance: Bridging Language Barriers in Indian FinTech,” arXiv:2512.01439.
- [6] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” in Proc. EMNLP 2023 (Demonstrations), 2023.
- [7] “Bridging the Safety Gap: A Guardrail Pipeline for Trustworthy LLM Inferences,” arXiv:2502.08142, 2025.
- [8] “Protect: Towards Robust Guardrailing Stack for Trustworthy Enterprise LLM Systems,” arXiv:2510.13351, 2025.
- [9] “CONSTRUCT: Real-Time Uncertainty Estimator for LLM Structured Outputs,” arXiv:2603.18014, 2026.
- [10] T. Baumgartner, B. Tai, L. Kaelin-Martin, C. Fan, L. Debaupte, B. Wang, and Y. Zhong, “VoiceCodeBench: Evaluating Exact Structured-Token Recovery in Automatic Speech Recognition,” arXiv:2608.28916, 2026.
- [11] C. Peyser, H. Zhang, T. N. Sainath, and Z. Wu, “Improving Performance of End-to-End ASR on Numeric Sequences,” arXiv:1907.01372, 2019.
- [12] C.-Y. Lee, C.-L. Li, T. Dozat, V. Perot, G. Su, N. Hua, J. Ainslie, R. Wang, Y. Fujii, and T. Pfister, “FormNet: Structural Encoding Beyond Sequential Modeling in Form Document Information Extraction,” in Proc. ACL 2022, 2022.
- [13] B. P. Majumder, N. Potti, S. Tata, J. B. Wendt, Q. Zhao, and M. Najork, “Representation Learning for Information Extraction from Form-like Documents,” in Proc. ACL 2020, 2020.
- [14] M. Aggarwal, H. Gupta, M. Sarkar, and B. Krishnamurthy, “Form2Seq: A Framework for Higher-Order Form Structure Extraction,” in Proc. EMNLP 2020, 2020.
- [15] D. Bohus and A. Rudnicky, “Constructing Accurate Beliefs in Spoken Dialog Systems,” Carnegie Mellon Univ., Tech. Rep., 2005.
- [16] S. Varges, S. Quarteroni, G. Riccardi, et al., “Investigating Clarification Strategies in a Hybrid POMDP Dialog Manager,” 2010.
- [17] J. Verhoeff, “Error Detecting Decimal Codes,” Mathematical Centre Tract 29, Mathematisch Centrum, Amsterdam, 1969.
- [18] National Payments Corporation of India, “Implementation of Verhoeff Algorithm by Banks for Aadhaar Related Applications,” NPCI circular.
- [19] H. Kanakia, S. Nadhamuni, and S. Sarma, “A UID Numbering Scheme,” white paper, Unique Identification Authority of India design contributors.
- [20] Unique Identification Authority of India, “UID Demographic Data Standards and Verification Procedure (DDSV) Committee Report v1.0,” UIDAI, 2010.